RAG CHEAT SHEET

Retrieve Smarter. Generate Better.

1. WHAT IS RAG?

RAG (Retrieval-Augmented Generation) combines retrieval of relevant information from an external knowledge base with the generative power of LLMs to produce accurate, up-to-date, and trustworthy answers.

In short:
RAG finds the right information first, then LLMs use it to generate better answers with source grounding.

2. LLMs vs RAG vs FINE-TUNING

Aspect	LLMs (Base)	RAG	Fine-Tuning
Knowledge Source	Trained data (up to cutoff)	External data (real-time)	Trained + custom data
Data Freshness	Limited (cutoff)	Up-to-date	Up-to-date after retraining
Customization	Low	High (use your data)	Very High
Cost	Low (inference)	Medium (infra + indexing)	High (training cost)
Time to Implement	Minutes	Days	Weeks+
Best For	General Q&A	Private data, enterprise search, real-time info	Task-specific behavior, style, domain

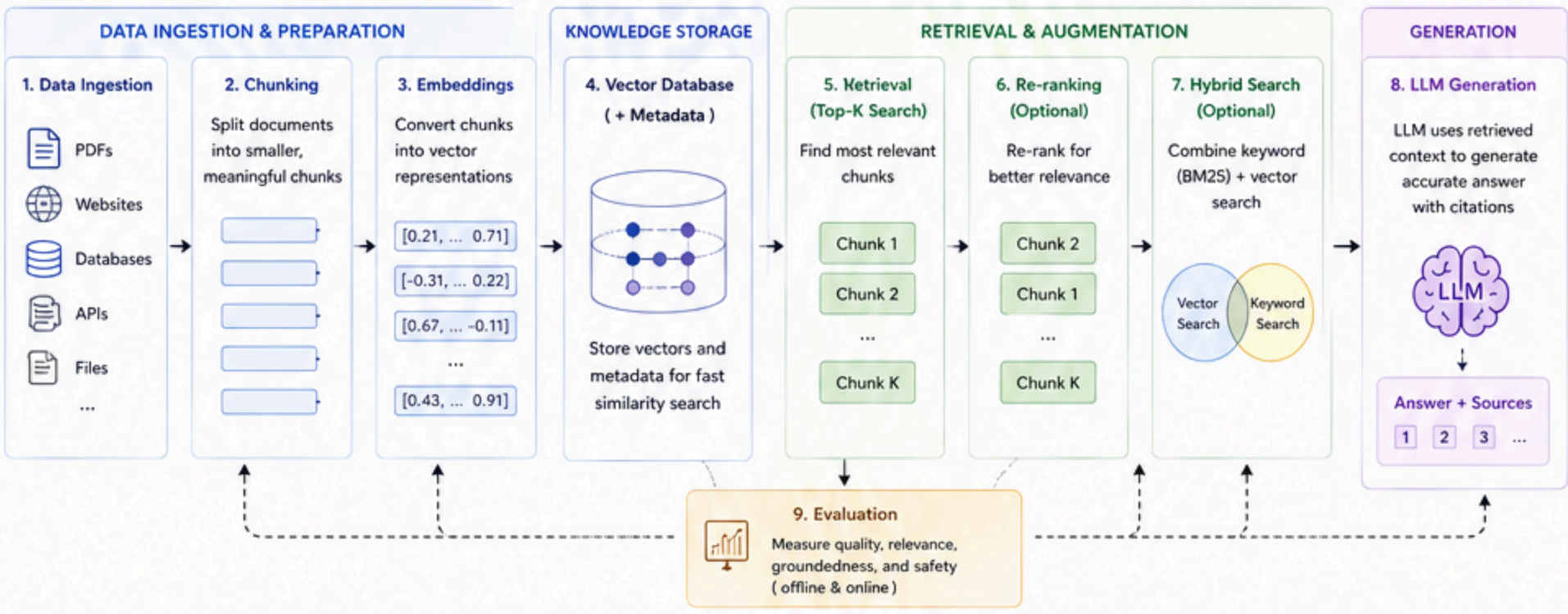
3. WHY DO WE NEED RAG?

- Access to latest and real-time information.
- Grounded answers with source citations.
- Reduces hallucinations.
- Leverages your private / internal data securely.
- Cost-effective compared to fine-tuning.
- Easier to update—just update your data.

4. BEST USE CASES

- Enterprise Search
- Technical Docs & Manuals
- Customer Support (FAQ Bots)
- Legal / Compliance Research
- Internal Knowledge Assistants
- Healthcare Information
- Document QA & Summarization
- Research & Market Intelligence

5. PRODUCTION-LEVEL RAG ARCHITECTURE



6. KEY COMPONENTS (AT A GLANCE)

Data Ingestion	Collect data from multiple sources.
Chunking	Split into optimal chunks (size, overlap).
Embeddings	Convert text into vector embeddings.
Vector DB	Store & index vectors for fast retrieval.
Retrieval	Fetch top-K semantically similar chunks.
Re-ranking	Improve relevance with cross-encoder or LLM.
Hybrid Search	Combine vector + keyword search for better recall.
LLM Generation	Generate answer using retrieved context.
Evaluation	Continuously evaluate quality & correctness.
Observability	Monitor, trace, and improve the system.

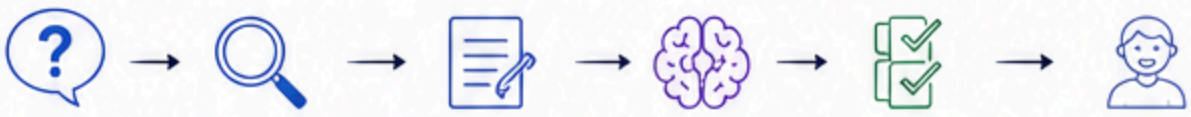
7. TIPS FOR BETTER RAG

- Use high-quality, domain-relevant data.
- Choose the right chunk size (e.g., 300–800 tokens).
- Use embedding model suited for your domain.
- Combine vector + keyword search.
- Re-rank results before sending to LLM.
- Always provide sources / citations.
- Evaluate continuously and iterate.
- Monitor latency, cost, and relevance.

8. COMMON METRICS

- **Retrieval Precision@K** — How many retrieved chunks are relevant.
- **MRR / NDCG** — Quality of ranked results.
- **Answer Faithfulness** — Is the answer supported by retrieved context?
- **Answer Relevance** — Does the answer solve the user's question?
- **Latency** — End-to-end response time.
- **Cost** — Tokens, storage, and infra cost.

9. RAG IN A NUTSHELL



REMEMBER

RAG is not just about retrieval or generation—it's about the right data, the right retrieval, and the right evaluation.

Great RAG = Great Data + Great Retrieval + Great Evaluation

DATA INGESTION TECHNIQUES

The First Step to Powerful RAG & AI Agents

1 WHAT IS DATA INGESTION?

Data ingestion is the process of collecting, extracting, loading, and preparing data from various sources so it can be used by AI/ML applications like RAG and Agents.

Example

A PDF manual is extracted, cleaned, chunked, embedded, and stored in a vector database for RAG.

2 HOW TO PREPARE DATA FOR RAG & AGENTS?

Extract	Clean	Chunk	Enrich (Optional)	Embed	Store/Index
Get data from sources	Remove noise, boiler-plate, extra spaces, etc.	Split into semantic chunks of optimal size	Add metadata (source, title, date, author etc.)	Convert chunks into embeddings	Store in Vector DB or Index

Best Practices

- Use meaningful chunk sizes (200–800 tokens typical)
- Keep metadata for filtering & better retrieval
- Clean data improves retrieval quality
- Normalize formats and handle duplicates

3 WHERE CAN DATA COME FROM?

- **Websites** — Public or private web pages
- **PDFs / Documents** — Manuals, reports, whitepapers
- **APIs** — REST / GraphQL endpoints
- **Databases** — SQL / NoSQL Databases
- **Email / Outlook** — EML, PST, Exchange
- **Cloud Storage** — S3, GCS, Azure Blob
- **Wikis** — Confluence, Notion, MediaWiki
- **Code Repositories** — GitHub, GitLab, Bitbucket
- **Spreadsheets** — Excel, CSV, Google Sheets
- **Others** — Logs, Output files, Presentations, etc.

4 PYTHON PACKAGES FOR DATA EXTRACTION

Task	Packages	Description
Web Scraping	requests, BeautifulSoup, Selenium, Playwright	Extract data from web pages (static & dynamic)
PDF Parsing	PyPDF2, pdfplumber, PyMuPDF (fitz)	Extract text, tables, metadata from PDFs
Docx / Office	python-docx, openpyxl	Read .docx, .xlsx, .pptx files
Text & Format	chardet, ftfy, regex	Detect encoding, fix text, clean content
Email	eml-parser, extract_msg	Parse emails and attachments
HTML to Text	readability-lxml, trafilatura	Extract main content from HTML
General Utilities	pandas, json, csv	Handle structured data

5 FRAMEWORKS FOR DATA LOADING & INGESTION

LANGCHAIN

Document Loaders

- WebBaseLoader (Load web pages)
- PyPDFLoader (Load PDFs)
- TextLoader (Load text files)
- CSVLoader / JSONLoader (Structured files)
- ConfluenceLoader (Load from Confluence)

Easily load, split, and process documents for RAG & Agents.

LLAMA INDEX (LlamaIndex)

Data Connectors (Readers)

- SimpleDirectoryReader
- PDFReader
- WebPageReader
- DatabaseReader
- APIReader
- WikipediaReader

Powerful indexing, transformations, and query engines for RAG & Agents.

6 HOW TO EXTRACT LIVE DATA (EXAMPLES)

A. Extract data from Wikipedia (Python)

```
import wikipedia
query = "Artificial Intelligence"
results = wikipedia.search(querry, results=3)
page = wikipedia.page(results[0])
print(page.title)
print(page.content[:1000])  # First 1000 chars
```

B. Extract content from a web page

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com/blog"
headers = {"User-Agent": "Mozilla/5.0"}

response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')

# Extract text
text = soup.get_text(separator=' ', strip=True)
print(text[:1000])
```

7 END-TO-END DATA INGESTION PIPELINE FOR RAG / AGENTS

8 CHUNKING STRATEGIES (DURING INGESTION)

- **Fixed Size Chunking** — Split by number of characters or tokens.
- **Recursive Chunking** — Split by sections, headings, then paragraphs.
- **Semantic Chunking** — Split based on semantic similarity.
- **Structure-Aware Chunking** — Respect headings, tables, lists, code blocks.

Tip: Choose chunk size based on use case (200–800 tokens is a good starting range).

9 METADATA TO CAPTURE

Metadata Field	Description
source	File path, URL, or system name
title	Document title or page title
author	Author / Owner
date	Creation / Published date
type	PDF, HTML, DOCX, etc.
tags	Category, labels
chunk_id	Unique chunk identifier
page / section	Page number or section title

10 TOOLS & PLATFORMS FOR INGESTION

Open Source

- **Apache Nifi** — Data flow automation
- **Airbyte** — Data ingestion from APIs & DBs
- **Dagster** — Data orchestration
- **Prefect** — Workflow orchestration
- **Unstructured** — Document partitioning
- **Tika** — Content extraction
- **Scrapy** — Web scraping framework

Managed / Cloud

- **AWS Glue** — Data catalog & ETL
- **Google Cloud Dataflow** — Ingestion pipelines
- **Azure Data Factory** — Data integration
- **Pinecone Ingestion** — Direct vector upserts
- **Weaviate Integrations** — Connectors
- **LangSmith** — Monitoring ingestion pipelines

11 EXAMPLE: INGEST PDF & LOAD INTO VECTOR DB (PYTHON)

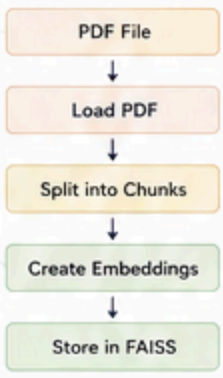
```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS

# 1. Load PDF
loader = PyPDFLoader("data/ai_whitepaper.pdf")
docs = loader.load()

# 2. Split
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# 3. Embed
embeddings = OpenAIEmbeddings()

# 4. Store in Vector DB
vectorstore = FAISS.from_documents(chunks, embeddings)
vectorstore.save_local("faiss_index")
```



12 COMMON CHALLENGES & SOLUTIONS

Challenge	Solution
Irrelevant / noisy data	Apply cleaning, filters, and content extraction
Large documents	Use smart chunking & summarize long sections
Dynamic websites	Use headless browsers (Selenium, Playwright)
Rate limits (APIs)	Implement retries, backoff, and caching
Duplicates	Deduplicate using hash or similarity checks
Encoding issues	Detect encoding and normalize (utf-8)

13 KEY TAKEAWAYS

- Good ingestion = High quality answers.
- Clean, well-chunked, and well-indexed data improves retrieval and saves cost.
- Capture rich metadata for better filtering and traceability.
- Automate and monitor your ingestion pipelines.
- Always validate your data and evaluate retrieval quality.

Remember:

Garbage In → Garbage Out

Better Data → Better AI

Strong data ingestion pipelines are the foundation of reliable RAG systems and intelligent agents.

METADATA TAGGING CHEAT SHEET FOR RAG & AGENTS

Better Context. Smarter Retrieval. Accurate Answers.

1 WHAT IS METADATA TAGGING?

Metadata tagging is the process of adding structured information (tags/attributes) to data or documents that describes the content, making it easier to filter, search, rank and retrieve relevant information in RAG & Agent systems.

EXAMPLE

Document: "Q4 Financial Results Report"

- source: "company_reports"
- doc_type: "financial_report"
- quarter: "Q4"
- year: "2024"
- department: "finance"
- region: "global"
- language: "en"
- sensitivity: "internal"
- created_at: "2024-02-10"

These tags help the system understand what the document is about and when/ where to use it.

2 WHY USE METADATA TAGGING?

- Improves retrieval relevance & precision
- Enables filtering by source, date, type, user, etc.
- Supports access control & data governance
- Enhances re-ranking with structured signals
- Helps in context routing for agents
- Improves explainability & traceability
- Reduces noise and hallucinations

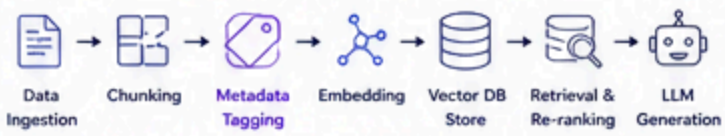
Example

Query: "What were our Q4 2024 expenses in EMEA?"

Without tags → many irrelevant docs

With tags (quarter=Q4, year=2024, region=EMEA, doc_type=financial_report) → precise results

3 WHERE METADATA FITS IN RAG & AGENTS



How it helps at each stage

- **Ingestion:** Tag data while extracting or importing
- **Retrieval:** Filter / boost results using metadata
- **Re-ranking:** Use metadata as additional features
- **Generation:** Provide context and citations with metadata

4 COMMON METADATA FIELDS (TAGS)

Category	Example Fields	Purpose
Source Info	source, author, url, doc_id	Track origin & traceability
Content Info	title, doc_type, summary, keywords	Understand content type
Temporal	created_at, updated_at, published_date, year, quarter	Time-based filtering
Organizational	department, team, owner, region, project	Access control & routing
Access & Security	visibility, sensitivity, roles, classification	Governance & security
Language & Format	language, file_type, encoding	Improve parsing & search
Usage Stats	views, downloads, feedback_score	Ranking & popularity
Custom / Domain	product, use_case, status, priority, customer_id	Domain-specific filtering

5 TYPES OF METADATA

Descriptive Metadata
Describes the content of the data.
e.g., title, summary, keywords, doc_type

Structural Metadata
Describes how data is structured.
e.g., section, heading, page_number

Administrative Metadata
Managing & governance related.
e.g., author, owner, created_at

Technical Metadata
Technical details about data.
e.g., file_type, size, encoding

Usage Metadata
How data is used.
e.g., views, feedback, last_accessed

6 WHERE TO STORE METADATA?

With Vector DB (Recommended)

Store metadata along with embeddings for fast filtering and retrieval.

Examples

- Pinecone (metadata field)
- Weaviate (properties)
- Qdrant (payload)
- Milvus (metadata)
- Chroma (metadata)

Use metadata filters during search (e.g., region = 'EMEA' AND year = 2024)

External Metadata Store

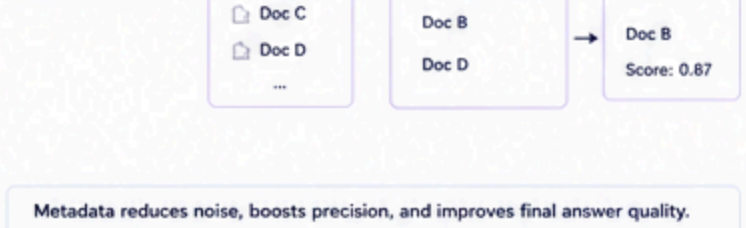
Store metadata in a separate DB and link with vector IDs.

Examples

- PostgreSQL / MySQL
- MongoDB
- DynamoDB
- Redis Hash
- ElasticSearch

Useful for complex queries, joins, analytics.

7 HOW METADATA IMPROVES RETRIEVAL (EXAMPLE)



Metadata reduces noise, boosts precision, and improves final answer quality.

8 HOW TO ADD / GENERATE METADATA?

Method	How it Works	Example
Manual Tagging	Human adds tags during upload	Add department, doc_type, etc.
Rule-Based	Use rules/regex to extract tags	Extract date, entity, file_type
LLM-Based Tagging	Use LLM to auto-generate tags/summary/keywords	LLM extracts topic, keywords, summary
Hybrid Approach	Combine rules + LLM + manual review	Better accuracy & consistency

9 EXAMPLE METADATA SCHEMA (JSON)

```
{
  "doc_id": "doc_12345",
  "title": "Q4 Financial Results Report",
  "source": "company_reports",
  "doc_type": "financial_report",
  "quarter": "Q4",
  "year": 2024,
  "region": "EMEA",
  "department": "finance",
  "author": "John Doe",
  "created_at": "2024-02-10T10:30:00Z",
  "language": "en",
  "sensitivity": "internal",
  "keywords": ["revenue", "expense", "profit"],
  "summary": "Q4 2024 financial performance overview...",
  "tags": ["finance","quarterly", "report"]
}
```

10 METADATA DRIVEN FILTERING EXAMPLES

Time Filter
Find all docs from 2024
Filter: year = 2024

Source Filter
Only from "HR_Portal"
Filter: source = "HR_Portal"

Type Filter
Only policies & guidelines
Filter: doc_type IN ["policy", "guideline"]

Access Control
Only for managers
Filter: visibility = "manager"

Geography Filter
Only for EMEA region
Filter: region = "EMEA"

Compound Filter
Filter: year = 2024 AND doc_type = "report" AND department = "finance"

11 METADATA IN AGENTIC SYSTEMS

Routing
Route query to right tool/knowledge base using metadata.

Tool Selection
Agent selects tools based on document capabilities (e.g., pdf, api, db).

Memory Management
Use metadata to store and retrieve agent memory/context.

Personalization
Use user_id, preferences, role to personalize responses.

12 TOOLS & LIBRARIES FOR METADATA

Tagging / Extraction

- LangChain (Document.metadata)
- LlamaIndex (Node.metadata)
- Unstructured (auto metadata)
- spaCy (NER & extraction)
- OpenAI / LLM (auto tagging)

Storage / Management

- Pinecone (metadata filtering)
- Weaviate (properties)
- Qdrant (payload)
- PostgreSQL / MySQL
- MongoDB
- Redis / RedisJSON
- ElasticSearch

13 METADATA BEST PRACTICES

- Define a standard metadata schema early
- Keep metadata consistent and normalized
- Use controlled vocabulary for tags
- Avoid too many unnecessary tags
- Validate metadata quality regularly
- Use metadata for filtering first, then semantic search
- Combine metadata + embeddings for best results
- Monitor and update metadata over time

14 BENEFITS SUMMARY

- Better Retrieval
- Higher Precision
- Reduced Hallucinations
- Faster Querying
- Access Control & Security
- Context Routing
- Personalized Responses
- Scalable & Maintainable

15 Common Challenges & Solutions

Challenge	Solution
Inconsistent Tags	Use schema, validation & controlled vocabulary
Missing Metadata	Use auto-tagging (LLM/rules) + manual review
Too Many Tags	Keep only useful & high-impact fields
Poor Quality Tags	Regular audits & feedback loop
Metadata Storage Overhead	Store only important metadata in Vector DB

16 END-TO-END EXAMPLE FLOW

Good metadata = better context understanding + accurate retrieval + reliable answers

17 QUICK CHECKLIST

- Have you defined your metadata schema?
- Are you tagging documents consistently?
- Are you using metadata filters in retrieval?
- Is metadata stored and indexed properly?
- Are you monitoring metadata quality?

Tag Smart. Retrieve Better. Answer Accurately. | Great Metadata = Great RAG & Smarter Agents.

CHUNKING STRATEGIES CHEAT SHEET

✨ Smarter Chunks. Better Retrieval. Stronger AI Answers. ✨

1. WHAT IS CHUNKING?

Chunking is the process of breaking down large documents into smaller, meaningful pieces (chunks) so that AI systems can understand, store, and retrieve them effectively.

2. WHY DO WE NEED CHUNKING?

- LLMs and embedding models have limit on input size.
- Smaller chunks are easier to embed and index.
- Improves accuracy by retrieving only relevant information.
- Reduces noise and irrelevant context.
- Saves storage and compute.

3. HOW CHUNKING IMPROVES RETRIEVAL IN RAG SYSTEMS

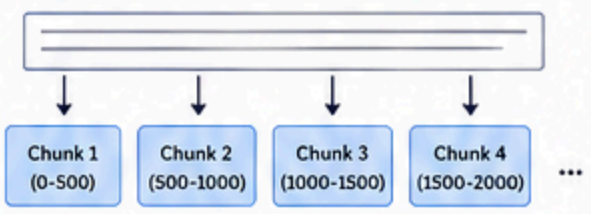
User Question	The question is converted to a vector (meaning).
Retrieve Top Relevant Chunks	Well-chunked content increases the chance of finding the right pieces.
LLM Generates Answer	The model gets focused, relevant context.
Better, Accurate Answer	Leads to more accurate, trustworthy answers.

CHUNKING STRATEGIES CHEAT SHEET

1 FIXED SIZE CHUNKING

How it works

The document is split into chunks of the same size based on character count or token count.



Why this strategy?

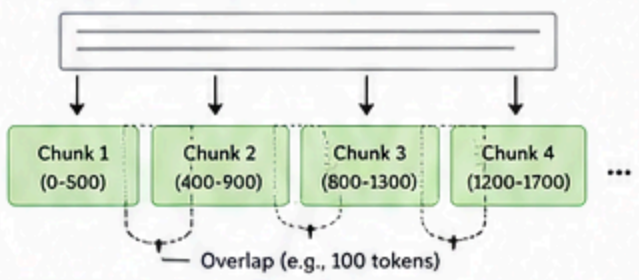
- Simple and fast to implement.
- Works well for structured or uniform content.
- Good baseline approach.

Best for: Documents with consistent structure and length.

2 FIXED OVERLAPPING CHUNKING

How it works

Chunks are created with a fixed size and a fixed overlap between consecutive chunks.



Why this strategy?

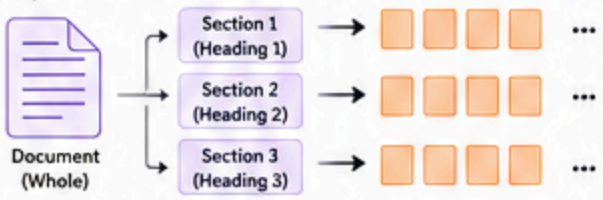
- Preserves context across chunk boundaries.
- Reduces chances of missing important info.
- Improves retrieval recall.

Best for: General-purpose use, especially for long-form content.

3 HIERARCHICAL CHUNKING

How it works

The document is split into larger sections first (e.g., by headings), then each section is further split into smaller chunks.



Why this strategy?

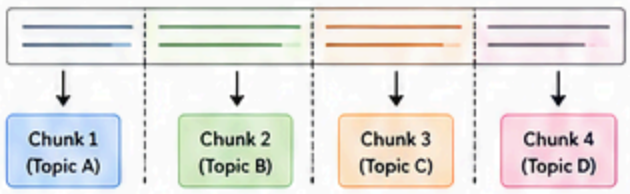
- Respects document structure.
- Good for retrieval at multiple levels (section or detail).
- Helps in building structured indexes.

Best for: Documents with clear headings and sections.

4 SEMANTIC CHUNKING

How it works

Chunks are created based on semantic meaning. The content is split where the topic or context changes.



Why this strategy?

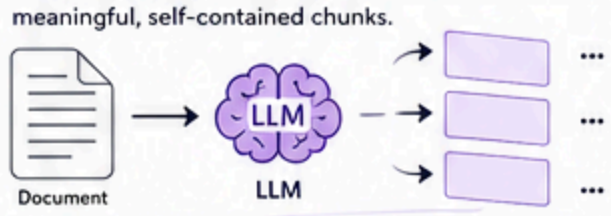
- Maintains meaning and coherence.
- Better retrieval relevance.
- Ideal for diverse and unstructured content.

Best for: Content with natural topic/idea boundaries (e.g., articles, blogs, reports).

5 LLM-BASED CHUNKING

How it works

An LLM is used to intelligently split the document into meaningful, self-contained chunks.



"Split this document into meaningful chunks that preserve context and are concise."

Why this strategy?

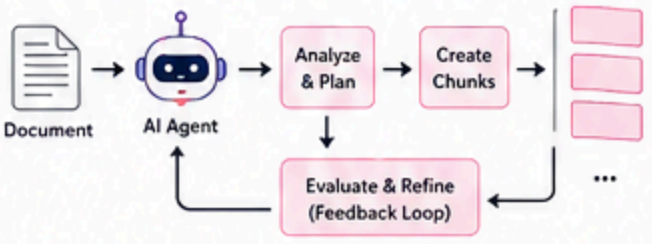
- Understands context like a human.
- Produces high-quality chunks.
- Great for complex or nuanced content.

Best for: High-quality retrieval where accuracy matters most.

6 AGENTIC CHUNKING

How it works

An AI agent analyzes the document, decides the best chunking strategy, and iteratively refines the chunks.



Why this strategy?

- Adaptive and context-aware.
- Can combine multiple strategies.
- Continuously improves chunk quality.

Best for: Complex, large-scale, or constantly changing content.

BEST PRACTICES

Know Your Use Case
Choose the strategy that fits your content type and goal.

Tune Chunk Size
Experiment with different sizes (e.g., 200–1000 tokens).

Evaluate Retrieval
Test which strategy yields the most relevant results.

Use Overlap
Use overlap when context continuity is important.

Prioritize Quality
Better chunks lead to better AI answers.

Monitor & Iterate
Continuously monitor and refine your chunking approach.

⭐ **Great chunks are the foundation of great AI answers.** Choose smart. Chunk smart. Build smarter AI systems.

EMBEDDINGS CHEAT SHEET

✦ Turn Text into Meaning: The Foundation of Modern AI ✦

1 WHAT ARE EMBEDDINGS?

Embeddings are numerical representations of text (or other data) in a continuous vector space. They capture meaning, context, and relationships between words, sentences, or documents.

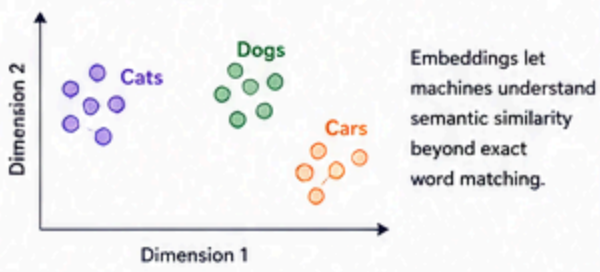
EXAMPLE

"The cat sat on the mat."

"A kitten rested on the rug." → [0.21, -0.37, 0.86, ... , -0.12]

[0.19, -0.35, 0.81, ... , -0.10]

Similar sentences have similar embeddings (vectors are close in space).



Embeddings let machines understand semantic similarity beyond exact word matching.

2. 2. EMBEDDINGS COME FROM TRANSFORMER ENCODER MODELS

Most modern embedding models use a Transformer Encoder to understand context and produce high-quality vector representations.

Text (Input) → Transformer Encoder → Vector Embedding (Output)

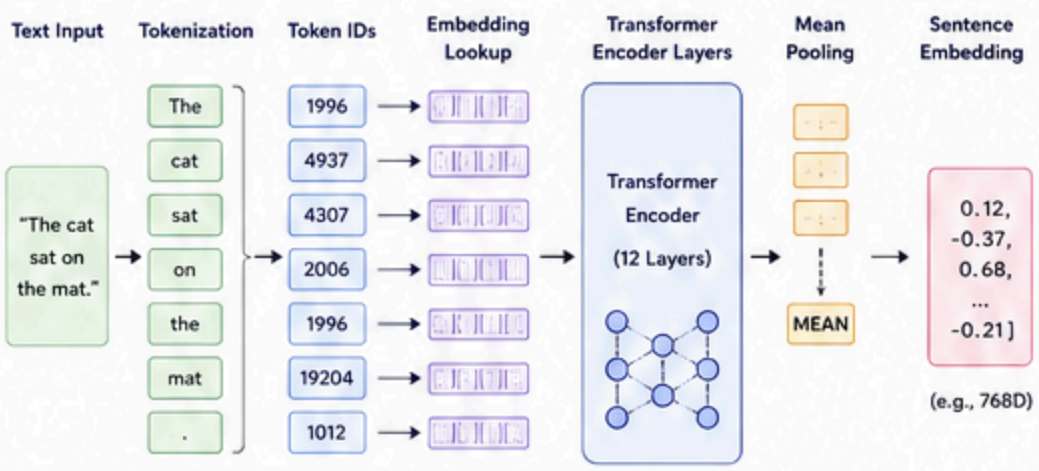
Why Transformer Encoder?

- Understands the meaning using self-attention.
- Captures context from both directions.
- Scales well with large data.
- Produces rich, general-purpose representations.

3. 3. HOW AN EMBEDDING MODEL WORKS (END-TO-END PIPELINE)

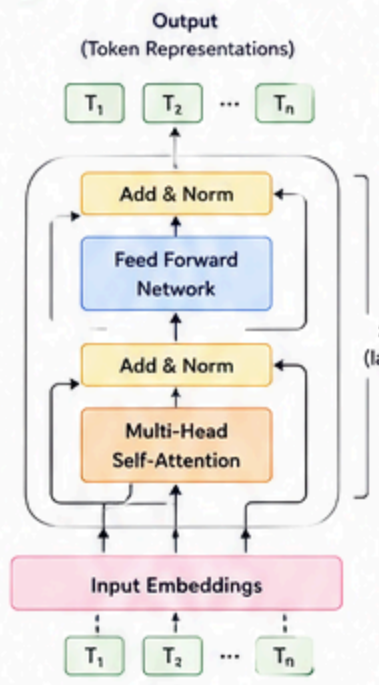
1. Text Input	Raw text (sentence, paragraph, or document) is provided.
2. Tokenization	Text is split into tokens (subwords/words).
3. Token IDs	Tokens are converted into numeric IDs.
4. Embedding Lookup	Each token ID is mapped to a dense vector.
5. Transformer Encoder Layers	Self-attention layers capture context and relationships.
6. Mean Pooling	Token vectors are averaged to get a single fixed-size vector.
7. Sentence Embedding (Output)	A single vector represents the entire input sentence.

4. EMBEDDING MODEL WORKFLOW (DETAILED)



- **Tokenization:** Break text into tokens (subwords).
- **Token IDs:** Convert tokens to numeric IDs.
- **Embedding Lookup:** Map IDs to dense vectors.
- **Transformer Encoder:** Understand context with self-attention.
- **Mean Pooling:** Average token vectors to get a single sentence vector.
- **Output:** Fixed-size embedding representing the input.

5. .. TRANSFORMER ENCODER ARCHITECTURE



Multi-Head Self-Attention
Allows each token to attend to all other tokens and capture relationships.

Add & Norm
Adds residual connection and layer normalization for stable training.

Feed Forward Network
Applies non-linear transformation to enrich representations.

Stacked L Layers
(typically 6, 12, 24, or more)
Deeper models capture more complex patterns.

6. TYPES OF EMBEDDING MODELS

A. OPEN SOURCE MODELS

Sentence Transformers (by UKPLab)
- State-of-the-art sentence embeddings.
- Easy to use, many pre-trained models.

Examples: all-MiniLM-L6-v2, all-mpnet-base-v2, e5-large-v2

Hugging Face Embedding Models
- Wide variety of models on Hugging Face Hub.
- Can be fine-tuned for your data.

Examples: BGE, GTE, Instructor, E5, DeBERTa-v3, and more

B. PROPRIETARY MODELS (CLOUD)

OpenAI Embeddings (text-embedding-3-small, text-embedding-3-large)
- High quality, scalable, easy via API.
- Strong performance across tasks.

Google Embedding Models (text-embedding-004)
- State-of-the-art performance.
- Multilingual and scalable.

Anthropic Claude Embeddings (coming via Claude API)
- High-quality embeddings integrated in Claude ecosystem.
- Great for retrieval + RAG.

Choose based on: quality, cost, latency, language support, and privacy needs.

7 EMBEDDING DIMENSIONALITY

The number of values in an embedding vector is called its dimensionality (e.g., 384D, 768D, 1536D, 3072D).

Why it matters?
- Higher dimensions → capture more nuance.
- Lower dimensions → faster & cheaper.
- Right balance → best cost vs performance.

COST vs PERFORMANCE TRADE-OFF

Dimension (Example)	Quality	Speed	Storage Cost	Use Case
384D	★★★☆☆	★★★★★	★★★★★	Fast retrieval, mobile, edge
768D	★★★★☆	★★★★☆	★★★★☆	General purpose (RAG, search)
1536D	★★★★	★★★☆☆	★★☆☆☆	High accuracy retrieval
3072D+	★★★★★	★☆☆☆☆	★☆☆☆☆	Complex tasks, high precision

Tip: Start with 768D–1536D for most use cases. Downsize if you need speed/cost, upsize for quality.

8. WHEN EMBEDDINGS FAIL

Embeddings are powerful, but not perfect.

- **Out of Domain** — Poor performance on topics far from training data.
- **Very Short Text** — Minimal context leads to weak representations.
- **Ambiguous or Sarcasm** — Nuance can be lost in vector representation.
- **Different Languages (Without Support)** — Mixed or unsupported languages reduce quality.
- **Heavily Technical / New Entities** — New terms or jargon may not be well represented.
- **Over-Compression** — Too low dimensions can lose important meaning.

Always evaluate with your own data and use retrieval + re-ranking for best results.

9. COMMON USE CASES

Semantic Search — Find meaning, not just keywords.

RAG Systems — Retrieve relevant context for LLM answers.

Clustering — Group similar documents or customers.

Recommendation Systems — Find similar items or users.

Classification — Use embeddings as input features.

Embeddings are the bridge between human language and machine understanding.
Better embeddings → Better retrieval → Better AI outcomes.

VECTOR DATABASE (VECTOR DB) CHEAT SHEET

Store, Index and Search Embeddings at Scale

1 WHAT IS A VECTOR DATABASE?

A Vector Database (VectorDB) is a specialized database designed to store, index and search high-dimensional vectors (embeddings). It enables fast similarity search using algorithms optimized for vectors.

EXAMPLE

Text	Embedding (Model)	Vector
"The cat sits on the mat"	→	[0.12, -0.45, 0.33, ..., 0.91] (1536 dims)

VectorDB stores the vector and allows finding similar vectors (semantic search).

2 VECTOR DB vs TRADITIONAL DB

Feature	Traditional DB (MySQL, Postgres)	Vector DB (Pinecone, Qdrant)
Data Type	Structured (rows/columns)	Vectors (embeddings)
Query	Exact match, filters, aggregations	Similarity search (approximate)
Search Type	Keyword / Exact	Semantic / Nearest Neighbor
Indexing	B-Tree, Hash	HNSW, IVF, Flat, ANN
Use Cases	Transactions, analytics	RAG, NLP, Recsys, Semantic Search
Example Query	WHERE name = 'John'	Find vectors similar to v (top_k)

3 HOW VECTOR DB WORKS (HIGH LEVEL)

Query Flow (in detail)

1. User query → converted to embedding
2. VectorDB searches for nearest neighbors
3. Returns Top-K results with metadata
4. Application uses results (e.g., in RAG)

Key Benefits

- Semantic understanding
- Handles unstructured data
- Scalable similarity search
- High performance with ANN index

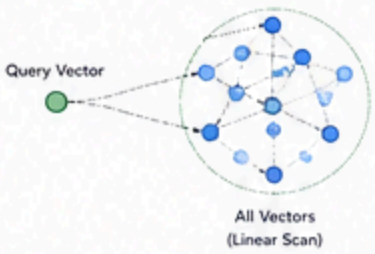
4 INDEXING TYPES IN VECTOR DBs

1. FLAT (Brute Force)

Stores all vectors. Computes distance with every vector during search.

How it works

- No complex index
- Exact search
- Good for small datasets



Complexity: O(N*d)

Use when: Dataset is small (< 10K vectors) or high recall is required.

2. IVF (Inverted File Index)

Clusters vectors using k-means. Search only in relevant clusters.

How it works

- Build coarse clusters (centroids)
- Assign vectors to clusters
- Search only in top-n clusters

Complexity: ~O((N/k + n) * d)

Use when: Medium to large datasets (millions of vectors).

3. HNSW (Hierarchical Navigable Small World)

Graph-based index. Efficient for high recall and low latency.

How it works

- Build multi-layer proximity graph
- Top layer: sparse, high-level links
- Bottom layer: dense connections
- Search by navigating the graph

Complexity: ~O(log N)

Use when: Large datasets requiring high recall and fast search.

4. ANN (Approximate Nearest Neighbor)

General term for techniques that find approximate nearest neighbors faster than exact search.

Includes:

- HNSW
- IVF
- PQ (Product Quantization)
- ScaNN, DiskANN, etc.

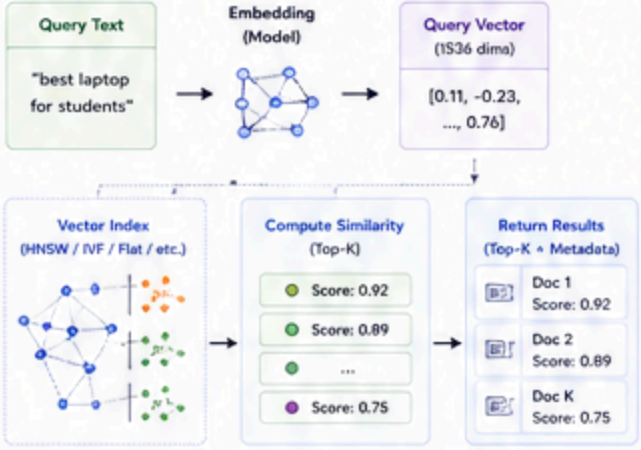
Trade-off: Higher speed ↔ Lower accuracy

5 SIMILARITY / DISTANCE METRICS

Metric	Formula	Use Case	Range
Cosine Similarity	$\text{sim}(A,B) = \frac{A \cdot B}{\|A\| \, \|B\|}$	Text embeddings (OpenAI, BERT)	[-1, 1] (1 is best)
Euclidean Distance	$d(A,B) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$	General purpose	[0, ∞) (0 is best)
Dot Product	$A \cdot B = \sum_{i=1}^{n} A_i B_i$	When vectors are already normalized	(-∞, ∞)
Manhattan Distance	$d(A,B) = \sum_{i=1}^{n} \|A_i - B_i\|$	Sparse / high-dimensional data	[0, ∞)
Pearson Correlation	$\rho_{A,B} = \frac{\text{cov}(A,B)}{\sigma_A \sigma_B}$	Statistical similarity	[-1, 1]

Cosine Similarity is the most commonly used in NLP & RAG systems.

6 HOW SIMILARITY SEARCH WORKS INSIDE VECTOR DB



The index reduces the search space. Similarity metric ranks the candidates and returns top-k results.

7 HOW VECTOR DB STORES DATA

Field	Description	Example
id / _id	Unique identifier	doc_123
embedding	Vector (float array)	[0.12, -0.45, ...]
text / content	Original text / document	"The cat sits on the mat."
metadata	Additional information	{"source": "blog", "author": "John"}
created_at	Timestamp	2024-05-20T10:00:00Z

Most DBs store:

- Vector (required)
- Metadata (recommended)
- Raw text (recommended)
- ID (required for retrieval)

Storage Options

- In-memory (fast)
- Disk-based (large scale)
- Hybrid (memory + disk)

8 POPULAR VECTOR DATABASES

Vector DB	Type	Index Support	Best For
Pinecone	Managed (Cloud)	HNSW, IVF, PQ	Production RAG, Scale
Weaviate	Open Source / Cloud	HNSW, PQ, BM25	Hybrid Search, RAG
Qdrant	Open Source / Cloud	HNSW, Payload Index	RAG, Filters, Speed
Milvus	Open Source / Cloud	HNSW, IVF, DiskANN	Large Scale, Enterprise
Chroma	Open Source	HNSW (default)	Developers, RAG
LanceDB	Open Source	IVF, PQ	Embeddings + SQL
Redis (RediaVL)	In-memory / Cloud	HNSW, Flat	Low latency, Real-time
pgvector	Postgres Extension	IVFFlat, HNSW	SQL + Vector Search

9 HOW TO CHOOSE THE RIGHT VECTOR DB?

Dataset Size	Small → Chroma, pgvector Large → Pinecone, Milvus, Qdrant
Performance Need	Low latency → Redis, Qdrant High recall → Pinecone, Weaviate
Filtering / Metadata	Need rich filters → Qdrant, Weaviate
Deployment	Managed → Pinecone, Weaviate Cloud Self-hosted → Milvus, Qdrant
Budget	Open Source → Milvus, Weaviate, Qdrant Paid / Managed → Pinecone
Use Case	RAG / Semantic Search / Recsys / Agents

10 BEST PRACTICES

- Normalize embeddings (especially for cosine similarity).
- Choose the right index based on dataset size and recall needs.
- Store useful metadata for filtering (source, timestamp, tags).
- Use hybrid search (vector + keyword) for better results.
- Tune index parameters (efConstruction, efSearch for HNSW).
- Monitor recall vs latency trade-off.
- Version your embeddings and re-index when model changes.

11 ADVANCED TOPICS

Quantization (PQ)

Compress vectors to reduce memory and increase speed.

Hybrid Search

Combine BM25 + Vector search for better results.

Re-ranking

Re-rank top-k results with cross-encoder / LLM.

Sharding

Distribute data across multiple nodes.

Common Index Parameters (HNSW)

- M (out-degree): 16–64 (higher = better recall, more memory)
- efConstruction: 100–512 (higher = better index quality)
- efSearch: 20–200 (higher = better recall, slower search)

12 COMMON USE CASES

- RAG (Retrieval Augmented Generation)
- Semantic Search
- Recommendation Systems
- Image / Video Search
- Anomaly Detection
- Conversational AI / Agents

13 KEY TAKEAWAYS

- Vector DBs store and search embeddings, not raw text.
- Index type selection impacts speed, cost, and accuracy.
- Similarity metrics define how "close" vectors are.
- Good indexing + good embeddings = great results.
- Always evaluate recall, latency, and cost together.

Right Embeddings + Right Index + Right Metric = Powerful Semantic Search

HYBRID SEARCH

1. WHAT IS HYBRID SEARCH?

Hybrid search combines semantic search (vector search) with keyword search (lexical search) to retrieve the most relevant results by leveraging the strengths of both approaches.

EXAMPLE

Query: "apple laptop with M2 chip"

Vector Search (Semantic)
Understands meaning

Top Results:
1. MacBook Air M2
2. MacBook Pro M2
3. Apple laptop 2024

Keyword Search (Lexical)
Matches exact terms

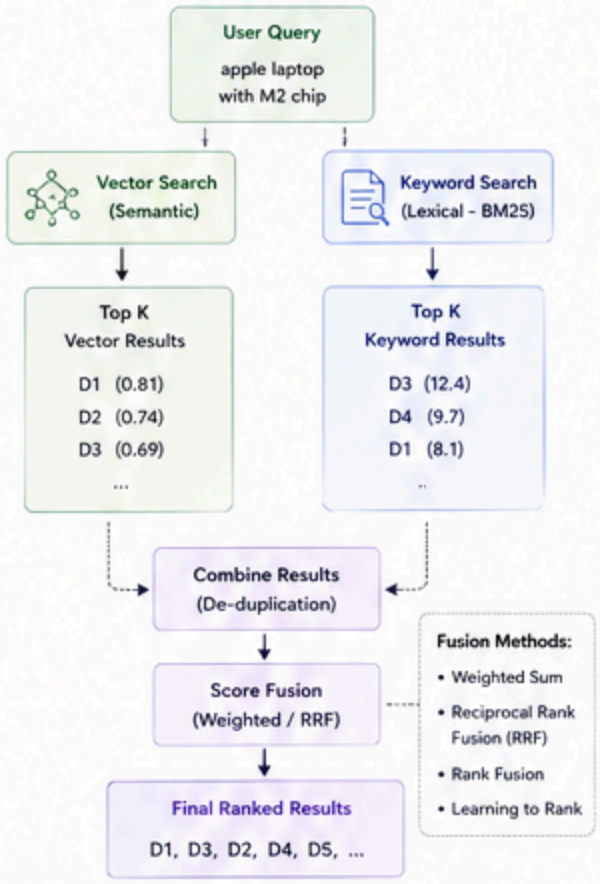
Top Results:
1. Apple M2 chip specs
2. Apple M2 release date
3. MacBook Air M2 price

Hybrid Search (Combined)
Better overall results

1. MacBook Air M2
2. MacBook Pro M2
3. Apple M2 chip specs
4. Apple laptop 2024
5. MacBook Air M2 price

2. HOW HYBRID SEARCH WORKS

Hybrid search runs vector search and keyword search in parallel, then combines and ranks the results.



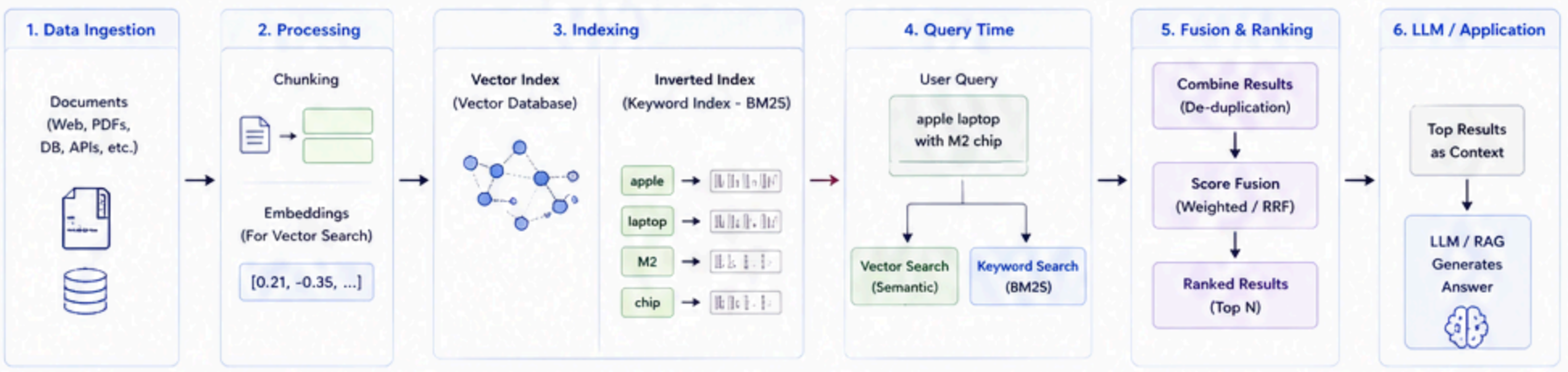
3. VECTOR SEARCH vs KEYWORD SEARCH

Aspect	Vector Search (Semantic)	Keyword Search (Lexical)
Matches	Meaning & context	Exact terms
Understands Synonyms	Yes	No
Handles Typos	Well	Poor
Best For	Conceptual, natural language queries	Specific facts, codes, names, numbers
Example Query	apple laptop	"Apple M2 release date"
Example Strength	Finds MacBook, iPad, Apple notebooks	Finds exact pages with M2 release date
Limitations	May miss exact keyword matches	Misses semantic similarity

4. WHY WE NEED HYBRID SEARCH FOR AI PROJECTS?

- Improves retrieval quality and recall.
- Handles both natural language and exact-match queries.
- Reduces hallucinations by retrieving more relevant context.
- Robust to query variations, typos, and ambiguity.
- Works better across different data types and domains.
- Boosts answer accuracy in RAG and AI applications.

5. HYBRID SEARCH ARCHITECTURE



6. BM25 (BEST MATCH 25) – HOW IT WORKS

BM25 is a probabilistic ranking function used in keyword search to estimate the relevance of a document to a query based on term frequency, document frequency and document length.

BM25 Scoring Formula

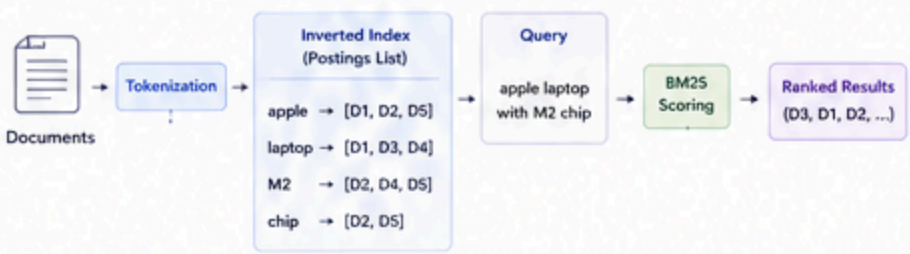
$$score(D, Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{avgdl}\right)}$$

Where:
- n = number of query terms
- $f(q_i, D)$ = frequency of term q_i in document D
- $IDF(q_i)$ = inverse document frequency of term q_i
- $|D|$ = length of document D
- $avgdl$ = average document length in corpus
- k_1, b = hyperparameters (typically k_1=1.2 to 2.0, b=0.75)

How BM25 Works (Steps)

1. Tokenize the query and documents.
2. Build an inverted index with term frequencies.
3. For each document, compute BM25 score for the query terms.
4. Rank documents by BM25 score (higher score = more relevant).

BM25 Architecture



7. SCORING & FUSION METHODS

Method	How It Works	Pros	Cons
Weighted Sum (Score Fusion)	Normalize scores from both searches and combine using weights.	Simple, effective	Needs proper weight tuning
Reciprocal Rank Fusion (RRF)	Uses ranks instead of raw scores. score = $\sum \frac{1}{k + rank}$	Robust, parameter-light	Less control over scores
Rank Fusion	Merge ranked lists and re-rank based on combined rank logic.	Easy to implement	May ignore score quality
Learning to Rank	Train a model on features (scores, ranks, metadada).	Best accuracy	Complex, needs training data

8. POPULAR TOOLS & DATABASES SUPPORTING HYBRID SEARCH

Databases / Search Engines

- **Elasticsearch** (BM25 + Vector Search)
- **OpenSearch** (BM25 + kNN)
- **Pinecone** (Dense Vector + Sparse)
- **Weaviate** (Hybrid Search)
- **Milvus** (BM25 + Vector Search)
- **Qdrant** (Dense + Sparse + RRF)

Frameworks / Libraries

- **LlamaIndex** (Hybrid Retriever)
- **LangChain** (Self Query + Hybrid Search)
- **Haystack** (Hybrid Search)
- **Django Haystack** (BM25 + Vector)
- **Vespa** (Sparse + Dense Hybrid)
- **Redis** (RedisSearch + Vector)

9. WHEN TO USE HYBRID SEARCH?

Scenario	Why Hybrid Search Helps
Natural language queries with specific terms	Captures meaning + exact matches
Queries with typos or variations	Vector handles variations, BM25 handles exact
Technical / domain-specific data	Combines conceptual + factual search
RAG for documents, knowledge bases	Improves context retrieval for LLM
Enterprise search	Better recall, precision and user satisfaction

10. BEST PRACTICES

- Tune weights or fusion method for your dataset.
- Normalize scores before fusion.
- Remove duplicates before final ranking.
- Use re-ranking (cross-encoder) after hybrid search for even better results.
- Monitor recall, precision and MRR.
- Choose top K carefully from both sides (50–200).

11. KEY TAKEAWAY

Hybrid search brings together the strengths of vector search (understanding meaning) and keyword search (exact matching) to deliver the most relevant and reliable results for AI applications.

Better retrieval → Better context → Better AI answers.

RERANKING CHEAT SHEET FOR RAG & AGENTS

Improve Relevance. Boost Accuracy. Deliver Better Answers.

1 WHAT IS RERANKING?

Reranking is the process of reordering initially retrieved documents/chunks using a more expensive (but more accurate) model to find the most relevant results for a query.

EXAMPLE

User Query	Initial Retrieval (Vector DB)	After Reranking
What are the benefits of AI in healthcare?	1. Doc A (score: 0.52) 2. Doc B (score: 0.47) 3. Doc C (score: 0.45) 4. Doc D (score: 0.40) 5. Doc E (score: 0.39)	1. Doc C (score: 0.93) 2. Doc A (score: 0.91) 3. Doc D (score: 0.88) 4. Doc B (score: 0.85) 5. Doc E (score: 0.40)

More relevant docs rise to the top!

2 WHY DO WE NEED RERANKING IN RAG?

- Vector search (ANN) is fast but not always accurate.
- Reranking improves precision@k and overall quality.
- Brings the most relevant context to the LLM.
- Reduces hallucinations and irrelevant answers.
- Essential for complex or nuanced queries.

Rule of Thumb: Retrieve more (e.g., top 50-100), Rerank, then pass top k (e.g., top 5-10) to the LLM.

3 WHERE DOES RERANKING FIT?

Typical Numbers

Retrieve: Top 50–200 → **Rerank:** Top 10–50 → **Pass to LLM:** Top 3–10

4 HOW RERANKING WORKS

Reranker takes the query and each retrieved document and outputs a relevance score.

Query

What are the benefits of AI in healthcare?

Retrieved from Vector DB (Top N)

- Doc 1: AI improves diagnosis ...
- Doc 2: History of healthcare ...
- Doc 3: AI reduces admin work ...
- Doc 4: Machine learning basics ...
- ...
- Doc N: Robotics in surgery ...

Reranker Model (Cross-Encoder / LLM / ColBERT etc.)

Query ↔	Doc 1	Score: 0.83
Query ↔	Doc 2	Score: 0.21
Query ↔	Doc 3	Score: 0.92
Query ↔	Doc 4	Score: 0.31
...		
Query ↔	Doc N	Score: 0.42

Reranked Results (Top M)

1. Doc 3 (0.92)
2. Doc 1 (0.83)
3. Doc N (0.42)
4. Doc 4 (0.31)
5. Doc 2 (0.21)

Reranker compares query with each document/chunk and predicts a relevance score. Higher score → more relevant.

5 RERANKING TECHNIQUES

Technique	How it Works	Pros	Cons	Best For
Cross-Encoder (BERT-based)	Encodes [query + document] together and predicts a relevance score.	High accuracy State-of-the-art for quality	Slow (expensive)	High quality reranking (Top 50-200)
LLM as a Judge (Prompt-based)	LLM reads query + doc and judges relevance (0-10 or Yes/No with reason).	Very flexible Understands semantics deeply	Expensive Slower	When you need explanations or complex logic
ColBERT	Late interaction model. Token-level embeddings with efficient scoring.	High accuracy Faster than Cross-Encoder	More complex to deploy	Large scale retrieval with high quality
SPLADE / Sparse Reranker	Learns sparse term weights and reranks using lexical signals.	Fast Scalable Interpretable	Slightly lower accuracy	Hybrid & large corpora
BM25 Rerank (Lexical)	Re-ranks based on keyword overlap (BM25 score).	Very fast No GPU needed	Limited semantic understanding	Keyword-heavy queries or as first rerank

6 SCORE CALCULATION (CROSS-ENCODER EXAMPLE)

Input: Query + Document → Model → Relevance Score (0 to 1)

Query

What are the benefits of AI in healthcare?

Document

AI can help in early disease detection and improve patient outcomes.

+ Cross-Encoder Model →

Score

0.92

The model jointly attends to both query and document tokens and predicts a relevance score (sigmoid or softmax).

7 RERANKING ARCHITECTURES

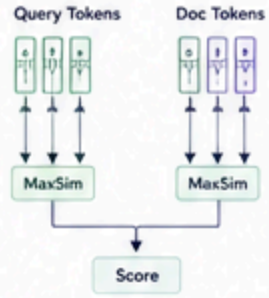
A. Cross-Encoder

Query + Document → Cross-Encoder (BERT) → Score

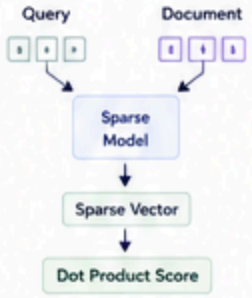
B. LLM as a Judge

Prompt (Query + Doc) Ask relevance (0-10) → LLM (GPT-4 / Llama etc) → Score / Reason

C. ColBERT (Late Interaction)



D. Sparse Reranker (SPLADE)



8 RERANKING PIPELINE (END-TO-END)

Example Flow

1. Retrieve Top 100 chunks from Vector DB
2. Rerank using Cross-Encoder → keep Top 10
3. Pass Top 5 to LLM for answer generation

9 WHEN TO RERANK?

- When initial retrieval returns many similar results.
- When query is complex or multi-intent.
- When high precision is critical (finance, legal, medical).
- When reducing hallucinations in RAG.
- When user satisfaction is more important than speed.

Skip Reranking if:

- You need ultra-low latency
- Corpus is very small
- Retrieval is already very accurate

10 TOOLS & LIBRARIES

Rerank Models / APIs

- Cohere Rerank (Cohere API)
- Jina Reranker (jina.ai)
- Voyage Rerank (voyage-ai)
- OpenAI Rerank (Coming Soon)

Open Source Models

- BAAI/bge-reranker-base
- BAAI/bge-reranker-large
- cross-encoder/ms-marco-MiniLM-L-6-v2
- ms-marco-MiniLM-L-12-v2
- mixedbread-ai/mxbai-rerank-large-v1

Libraries

- sentence-transformers
- rank-bm25
- pycolbert
- LlamaIndex
- LangChain

11 WHERE TO STORE RERANKING MODELS?

Self-Hosted (Open Source)

HuggingFace Models

Deploy with:

- SentenceTransformers
- vLLM / TGI
- ONNX / OpenVINO

Managed APIs

- Cohere Rerank
- Jina Reranker
- Voyage Rerank
- OpenAI (Future)
- Together AI

Caching Layer

Cache frequent (query, doc) → score pairs (Redis)

12 BEST PRACTICES

- Retrieve more, rerank fewer.
- Tune chunk size (200-500 tokens) for better results.
- Combine semantic (vector) + lexical (BM25) retrieval.
- Use hybrid reranking (e.g., Cross-Encoder + BM25 fallback).
- A/B test reranker models.
- Cache reranker scores for frequent queries.
- Monitor latency vs relevance trade-off.

13 KEY TAKEAWAYS

- Reranking improves the quality of retrieved context.
- It sits between retrieval and generation.
- Cross-Encoders give best accuracy, LLMs add flexibility.
- Use the right reranker for your latency & accuracy needs.
- Good reranking = Better answers, happier users.

QUICK COMPARISON TABLE

Method	Accuracy	Speed	Scalability	Cost	Notes
Cross-Encoder	★★★★★	★★☆☆☆	★☆☆☆☆	$$$	Best accuracy, slower
LLM as Judge	★★★★☆	★☆☆☆☆	★☆☆☆☆	$$$$	Very flexible, most expensive
ColBERT	★★★★☆	★★★★☆	★★★☆☆	$$	Good balance
SPLADE / Sparse	★★★☆☆	★★★★★	★★★★★	$	Fast & scalable
BM25 Rerank	★★☆☆☆	★★★★★	★★★★★	$	Lexical match only

EXAMPLE PROMPT (LLM AS A JUDGE)

You are a helpful relevance judge.

Query: (query)
Document: (document)

On a scale of 0 to 10, how relevant is this document to the query?
Return only the score and a short reason.

Score:
Reason:

COMMON METRICS

Precision@k	Higher is better
Recall@k	Higher is better
MRR	Higher is better
nDCG@k	Higher is better
MAP	Higher is better

Reranking is the secret sauce that turns good retrieval into great answers.

Retrieve smart. Rerank smarter. Answer best.

PROMPT ENGINEERING CHEAT SHEET
FOR RAG & AGENTS

1 WHAT IS PROMPT ENGINEERING?

Prompt engineering is the art and science of designing inputs (prompts) that guide LLMs to generate accurate, relevant, consistent, and useful outputs.

EXAMPLE

Poor Prompt

Q: Summarize this document.

Result: (Too generic, may miss key points)

Better Prompt

You are a helpful analyst.
Summarize the following document in 5 bullet points highlighting key findings and risks.

Document: <context>

Result: (Focused, structured, and relevant)

Key Takeaway

The better the prompt, the better the output.

2 TYPES OF PROMPTS

Prompt Type	Purpose
System Prompt	Sets the behavior, role, and guidelines for the model.
User Prompt	The actual request or question from the user.
Assistant Prompt	(Optional) Few-shot examples or previous assistant replies.
Context Prompt	External information, documents, or data provided for grounding.
Memory Prompt	Conversation history or memory from previous turns.
Tool Prompt	Instructions for using tools (e.g., calculator, search, API).
Guardrail Prompt	Constraints, safety rules, or content policies.

3 HOW TO DESIGN A PROMPT FOR RAG & AGENTS

A good prompt = Clear Instruction + Relevant Context + Output Format

Recommended Prompt Structure

Role	Define the role/persona for the model.
Goal	State the task or objective clearly.
Context	Provide relevant context (retrieved docs/data).
Instructions	Step-by-step guidance on what to do.
Constraints	Rules, tone, length, do's & don'ts.
Output Format	Specify how the answer should be structured.
Examples (opt.)	Provide examples for better accuracy.

RAG Prompt Template (Example)

```
System: You are a knowledge assistant. Answer based only on the
        provided context. If the answer is not in the context, say
        "I don't know".
Context:
        {retrieved_context}
User Question:
        {user_query}
Instructions: Provide a clear, concise, and well-structured answer.
              Cite sources if available.
Output Format: Markdown with headings and bullet points.
```

4 PROMPTING TECHNIQUES (WITH EXAMPLES)

Zero-Shot Prompting

Ask the model directly without examples.

Example:

Q: Translate to French: "Hello, how are you?"

Few-Shot Prompting

Provide a few examples to guide the model.

Example:

Q: Sentiment: I love it.
A: Positive

Q: Sentiment: Bad service.
A: Negative

Q: Sentiment: Excellent experience.

Chain-of-Thought (CoT)

Encourage step-by-step reasoning.

Example:

Q: If a train leaves A at 2 PM and travels 120 km/h for 3 hours, when does it arrive?

A: Let's think step by step...

Instruction Prompting

Use clear instructions for complex tasks.

Example:

Extract all dates from the given text and return them in YYYY-MM-DD format.

Persona Prompting

Give the model a persona for better tone & style.

Example:

You are a senior legal advisor. Explain this contract in simple terms.

Self-Consistency

Generate multiple reasoning paths and pick the most consistent answer.

Tree of Thoughts (ToT)

Explore multiple reasoning branches before making a decision.

ReAct (Reason + Act)

Combine reasoning with actions (e.g., search, tools) for better results.

Generated Knowledge Prompting

Ask model to generate background knowledge before answering.

Code / Structured Prompting

Use JSON, XML, tables for structured output.

5 HOW TO CONNECT WITH CONTEXT (IN RAG)

User Query → Retrieve Relevant Documents → Insert into Prompt → LLM Generates Grounded Answer

Best Practices

- Insert the most relevant chunks (Top-K) in the prompt.
- Clearly separate context and question.
- Tell the model to answer only from the context.
- Handle long context with summarization or reranking.
- Always cite sources when possible.

Context Formatting Tips

- Use clear delimiters: `"""`, `<context> </context>`, `---`
- Add source metadata: `[Source: doc.pdf, page 4]`
- Keep chunks small and focused (`200–800` tokens).

6 AGENT PROMPT DESIGN PRINCIPLES

Clarity
Be clear about the goal and expected behavior.

Decomposition
Break complex tasks into sub-tasks.

Tool Awareness
Explain available tools and how to use them.

Memory
Provide history, state, and intermediate results.

Safety & Guardrails
Add boundaries and content policies.

Agent Prompt Template

```
You are an intelligent assistant that can use tools
to help users.

Goals:
- Understand the user request
- Plan steps
- Use tools if needed
- Provide the best answer

Available Tools:
1. search(query): Search the web
2. calculator(expression): Do math
3. get_weather(city): Get weather info

User Request: {user_query}
Conversation History: {history}
Respond with your plan, tool calls (if any),
and final answer.
```

7 PROMPT DOS & DON'TS

DO'S	DON'TS
Be specific and clear	Be vague or ambiguous
Provide context and examples	Give conflicting instructions
Use structured format	Overload with unnecessary context
Set expectations	Ignore output format
Iterate and refine	Assume the model knows everything
Test with edge cases	Skip validation and testing

8 EXAMPLE: RAG PROMPT (FULL)

```
System: You are a helpful assistant for answering questions
based on provided documents. Do not use outside knowledge.
If the answer is not in the context, say "I don't know".

Context:
"""
[Source: annual_report_2024.pdf, page 12]
Our company revenue grew by 18% in 2024, reaching $2.48.
The growth was driven by strong performance in cloud
services and international markets.
"""

User Question: What was the revenue growth in 2024?

Instructions:
- Answer using the context above.
- Be concise.
- Cite the source.

Answer Format: Markdown.
```

9 TOOLS & FRAMEWORKS FOR PROMPT ENGINEERING

Libraries & Frameworks

- **LangChain** – Prompt templates, chains, RAG pipelines
- **LlamaIndex** – Prompt templates, RAG, context management
- **PromptFlow (Microsoft)** – Design, test and evaluate prompts
- **DSPy** – Programming framework for LM prompts
- **Guidance** – Structured prompting and validation
- **Haystack** – RAG pipelines and prompt orchestration

Prompt Engineering Tools

- **ChatGPT Playground** – Test and iterate prompts
- **Anthropic Console** – Test Claude prompts
- **OpenAI API Playground** – Test GPT prompts
- **PromptLayer** – Log, test, version prompts
- **Helicone** – Prompt observability
- **LangSmith** – Debug and evaluate prompts
- **TruLens** – Evaluate LLM outputs
- **DeepEval** – Evaluate RAG & LLM apps

10 EVALUATE YOUR PROMPTS

What to Evaluate?

- **Relevance** – Does it answer the question?
- **Faithfulness** – Is it grounded in context?
- **Correctness** – Is the information accurate?
- **Completeness** – Is it complete?
- **Coherence** – Is it well structured?
- **Safety** – Is it safe and appropriate?

Evaluation Methods

- Human evaluation
- LLM-as-a-judge
- Automated metrics (RAGAS, DeepEval, Trulens)
- A/B testing different prompts

11 QUICK REFERENCE CHECKLIST

- [x] Define role and goal
- [x] Provide relevant context
- [x] Give clear instructions
- [x] Add constraints and tone
- [x] Specify output format
- [x] Use examples if needed
- [x] Keep it concise and structured
- [x] Test, evaluate, and refine

Remember

Great prompts don't happen by accident.
They are designed, tested, and improved.

12 PROMPT ENGINEERING MINDSET

Understand the Task → Provide the Right Context → Guide the Model → Get the Best Output → Evaluate & Improve → Repeat & Scale

LLMS CHEAT SHEET FOR RAG & AGENTS

Choose Right Model. Prompt Right. Build Smarter.

1 WHAT ARE LLMS?

Large Language Models (LLMs) are advanced AI models trained on massive text data to understand and generate human-like text, reason, follow instructions, and use tools.

EXAMPLE

User: Explain RAG in simple terms.

LLM: RAG (Retrieval Augmented Generation) combines retrieval of relevant documents with LLM generation to produce accurate, context-aware answers.

2 ROLE OF LLMS IN RAG & AGENTS

- Understand user intent
- Generate queries for retrieval
- Summarize and re-rank content
- Synthesize context to generate answers
- Handle multi-turn conversations
- Use tools/APIs and take actions (Agents)
- Maintain memory and reasoning

Key Idea:

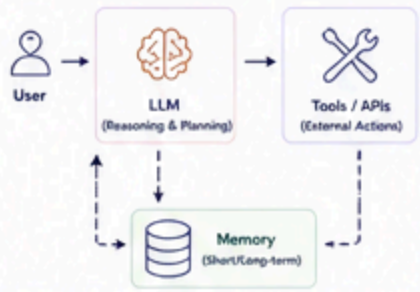
LLMs act as the brain that understands, reasons, and generates while other components provide context, memory and actions.

3 HOW LLMS FIT IN RAG PIPELINE

LLM is used for:

- Query understanding & rewriting
- Answer generation with retrieved context
- Summarization, classification, extraction, etc.

4 HOW LLMS FIT IN AGENTIC SYSTEMS



LLM Capabilities in Agents:

- Plan & break down tasks
- Use tools and APIs
- Maintain memory & context
- Reflect, evaluate & adapt

5 TYPES OF LLMS

Type	Description	Examples	Best For
General LLMs	General purpose text understanding & generation	GPT-4o, Llama 3, Claude 3, Gemini 1.5	Chat, QA, Writing
Instruction Tuned LLMs	Fine-tuned to follow instructions better	Llama 3 Instruct, Mistral Instruct, ChatGPT	RAG, Agents
Code LLMs	Specialized in code generation & reasoning	Code Llama, DeepSeek Coder, StarCoder2	Coding, Dev Agents
Domain LLMs	Trained for specific domains/industries	Med-PaLM, BioGPT, FinGPT	Healthcare, Finance
Multimodal LLMs	Understand text + images + other modalities	GPT-4o, Gemini 1.5, LLaVA	Vision, Docs, Charts

6 HOW TO CHOOSE THE RIGHT LLM?

Factor	Considerations
Use Case	RAG, Chatbot, Coding, Reasoning, Summarization, etc.
Quality	Accuracy, reasoning, hallucination rate
Context Window	4K, 16K, 32K, 128K (larger is better for long docs)
Cost	Input/Output token pricing
Speed / Latency	Response time & throughput
Tool Use / Function Calling	Does it support tools, JSON output, function calling?
Safety & Alignment	Content filters, biases, privacy
Deployment	API (cloud) vs Local (self-hosted)

7 POPULAR LLMS (QUICK REFERENCE)

Model	Provider	Context Window	Strengths	Best For
GPT-4o	OpenAI	128K	Powerful, multimodal, function calling	RAG, Agents, GPTs
GPT-4 Turbo	OpenAI	128K	High quality, fast	General, RAG
Claude 3 Opus	Anthropic	200K	Strong reasoning, safe outputs	Complex Q&A, Analysis
Gemini 1.5 Pro	Google	1M	Huge context, multimodal	Long docs, RAG
Llama 3 70B	Meta	128K	Open weights, cost-effective	Self-hosted, RAG
Mistral Large 2	Mistral AI	128K	High quality, fast & efficient	RAG, Agents
Mixtral 8x22B	Mistral AI	64K	MoE, cost-effective	Scale & Efficency

8 PROMPTING LLMS EFFECTIVELY

Prompt Structure (Recommended)

Role	Define the role of the LLM
Goal	State the task clearly
Context	Provide background information
Instructions	Step-by-step guidance
Constraints	Rules, tone, length, format
Output Format	How the answer should look
Examples (Opt.)	Few-shot examples

Best Practices

- Be clear and specific
- Break complex tasks into steps
- Use delimiters (""", <context>)
- Ask the model to think step-by-step (CoT)
- Provide examples for better results

9 PROMPTING TECHNIQUES

Zero-Shot
Ask directly without examples.

Few-Shot
Provide examples to guide the model.

Chain-of-Thought (CoT)
Encourage step-by-step ressoning.

Self-Consistency
Generate multiple reasoning paths and vote on answer.

Tree of Thoughts (ToT)
Explore multiple reasoning branches like a tree.

ReAct (Reason + Act)
Combine reasoning with action (using tools).

Generated Knowledge
Ask model to generate background knowledge before answering.

10 CONTEXT WINDOW & TOKEN MANAGEMENT

- LLMs have a limited context window (tokens).
- Manage tokens smartly for better performance.

Tips

- Chunk documents before retrieval
- Retrieve top-k relevant chunks
- Summarize long context
- Remove irrelevant/duplicated content
- Use compression & re-ranking
- Monitor token usage & cost

Token Example

1 token ≈ 4 characters in English

1K tokens ≈ 750 words

Always leave room for the model to generate output.

11 REDUCE HALLUCINATIONS

- Provide relevant and complete context
- Use system prompt to force grounded answers
- Ask: "If you don't know, say you don't know."
- Use citations / sources in output
- Use re-ranking & filtering
- Evaluate answer faithfulness
- Prefer domain-specific models when possible

Grounding Prompt Example

"Answer the question only using the provided context.
If the answer is not in the context, say "I don't know"."

Context:
(context)

Question: (question)

12 FUNCTION CALLING / TOOLS / API USE

Modern LLMs can call functions, tools, APIs and interact with external systems.

Common Tools

Search · Calculator · DB Query · Web APIs · Code Interpreter · File Reader

13 MEMORY IN LLMS (FOR AGENTS)

Short-Term Memory (Conversation History)

Stored in context window for current session.

Long-Term Memory (Persistent)

Stored in external systems for future use.

- Vector DB (Pinecone, Weaviate)
- SQL/NoSQL (Postgres, MongoDB)
- Graph DB (Neo4j)
- File/Blob Storage (S3, GCS)

14 EVALUATING LLMS

Key Metrics

- Relevance / Answer Quality
- Faithfulness (Grounded)
- Correctness
- Context Precision / Recall
- Hallucination Rate
- Latency
- Cost (Tokens)

- Human Evaluation
- LLM as a Judge
- Automatic Metrics (RAGAS, DeepEval)
- A/B Testing

15 LLMS DEPLOYMENT OPTIONS

Option	Description	Best For
Hosted API (OpenAI, Anthropic)	Use via API	Quick start, No infra
Self-Hosted Open Models (Llama, Mistral, Mixtral)	Run on your own infrastructure	Privacy, Cost control
Managed Services (AWS Bedrock, Azure OpenAI, GCP Vertex)	Fully managed LLM services	Enterprise Scale
Edge / On-device LLMs (Phi, Gemma, TinyLlama)	Run on device (low resource)	Mobile, Edge Apps

16 COST OPTIMIZATION TIPS

- Use the smallest model that works
- Cache responses where possible
- Use embeddings + retrieval instead of huge prompts
- Compress & summarize context
- Batch requests
- Monitor token usage
- Use open source models for non-critical tasks

17 POPULAR TOOLS & FRAMEWORKS

For RAG	For Agents	For Evaluation
LangChain	LangGraph	RAGAS
LlamaIndex	AutoGen	DeepEval
Haystack	CrewAI	TruLens
GPT Index	Phala Network	Phoenix
RAGFlow	LLMStack	Wandb Eval

18 USE CASES

- RAG Chatbots & Copilots
- Document Q&A & Search
- Enterprise Knowledge Assistants
- Data Analysis & Reporting
- Code Generation & Debugging
- Customer Support Automation
- AI Agents (Task Automation)

19 QUICK CHECKLIST

- Have you defined the right use case?
- Did you choose the right LLM?
- Is your prompt clear and structured?
- Is your context relevant and optimized?
- Are you grounding answers with context?
- Are you managing memory properly?
- Have you evaluated quality & latency?
- Are you tracking cost?

The Right LLM + The Right Context + The Right Prompt = Intelligent, Reliable & Scalable RAG & Agents

MEMORY & CACHING CHEAT SHEET
FOR RAG & AGENTIC SYSTEMS

1 WHAT IS MEMORY?

Memory is the ability of a system (RAG or Agent) to retain information from past interactions, documents, or experiences to improve future responses and decisions.

EXAMPLE

User: I'm planning a trip to Japan next month.

Agent (remembers later):
Your trip to Japan is next month. Do you want hotel recommendations?

2 WHAT IS CACHING?

Caching is storing results (responses, embeddings, retrievals, API calls, etc.) temporarily so they can be reused to reduce latency, cost, and load.

EXAMPLE

3 MEMORY vs CACHING

Aspect	Memory	Caching
Purpose	Retain information for future use	Speed up by reusing results
Duration	Short-term or Long-term	Short-term (temporary)
Content	Facts, context, user profile, history, documents, preferences	Responses, embeddings, retrieval results, API outputs
Use in	Continuity, personalization, learning, context retention	Performance, latency, cost reduction
Storage	DBs, Vector DBs, Files, Graph DBs	Redis, In-Memory, CDN, Local Cache
Eviction	Rarely (retain longer)	Often (TTL, LRU, size-based)

4 TYPES OF MEMORY

A. SHORT-TERM (Working Memory)

Stores recent conversation turns, intermediate results, and context within a session.

- Lives in the current session
- Helps maintain context window
- Lost when session ends

Examples: Chat history, tool results, intermediate reasoning steps

B. LONG-TERM (Persistent Memory)

Stores information across sessions and users for personalization and knowledge retention.

- Persists across sessions
- Used for personalization, facts,
- preferences, documents

Examples: User profile, past activites, knowledge base, documents

5 WHERE TO STORE MEMORY?

SHORT-TERM MEMORY
(Fast Access, Ephemeral)

- In-Memory (RAM) (e.g., Python dict, buffer)
- Redis (In-memory Store)
- Session Store (e.g., Flask session, FastAPI)

LONG-TERM MEMORY
(Persistent, Durable)

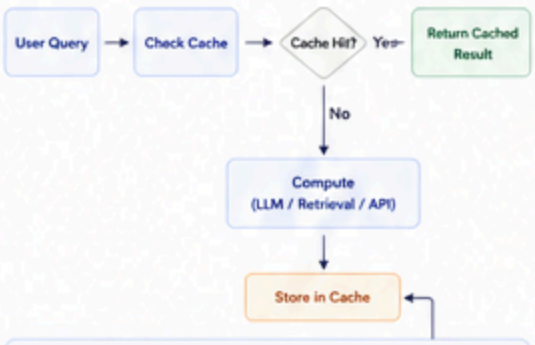
- Vector DB (Pinecone, Weaviate, Qdrant)
- Relational DB (PostgreSQL, MySQL)
- Document DB (MongoDB)
- Graph DB (Neo4j)
- File/Object Storage (S3, GCS, Azure Blob)

6 HOW MEMORY WORKS IN RAG & AGENTS

Memory Examples

- Remember user preferences (e.g., "I like vegan food")
- Recall past tool usage and outcomes
- Maintain conversation continuity
- Store important facts for future sessions

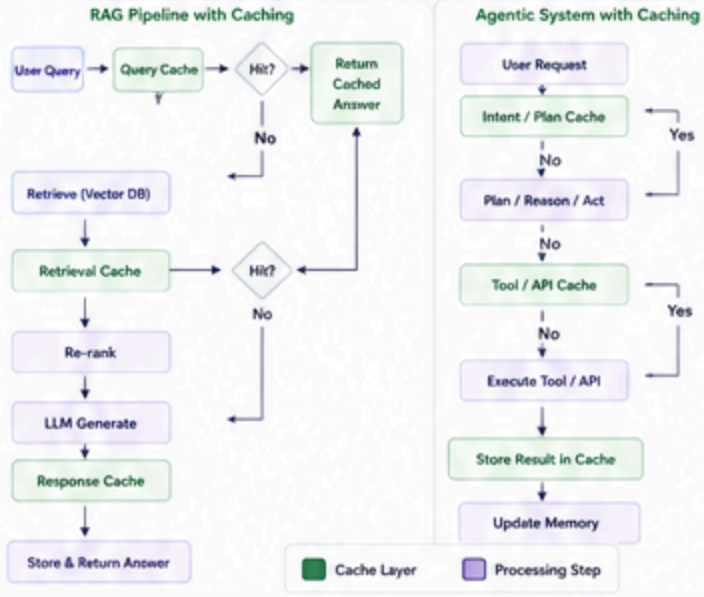
7 HOW CACHING WORKS



Cache Examples

- Cache LLM responses to common questions
- Cache embeddings of frequent texts
- Cache retrieval results for popular queries
- Cache tool/API responses (weather, search, etc.)

8 WHERE CACHING FITS IN RAG & AGENTIC SYSTEMS



9 CACHING LAYERS & WHAT TO CACHE

Layer	What to Cache	Examples / Tools
1. Query Level	Exact user query responses	Redis, Memcached
2. Retrieval Level	Top-K docs / chunks for a query	Redis, Disk Cache
3. Embedding Level	Text → Embedding results	Redis, In-Memory
4. LLM Response Level	Final LLM responses	Redis, Disk, CDN
5. Tool/API Level	External API or tool responses	Redis, HTTP Cache
6. Plan/Reasoning Level (Agents)	Plans, decisions, actions	Redis, DB

10 POPULAR TOOLS & TECHNOLOGIES

MEMORY STORAGE

- Vector DBs — Pinecone, Weaviate, Qdrant, Milvus
- Relational DBs — PostgreSQL, MySQL
- Document DBs — MongoDB, CouchDB
- Graph DBs — Neo4j, ArangoDB
- Object Storage — AWS S3, GCS, Azure Blob
- In-Memory Stores — Redis, Memcached

CACHING SOLUTIONS

- In-Memory Cache — Redis, Memcached
- Local Cache (Process) — functools.lru_cache, cachetools
- HTTP / CDN Cache — Cloudflare, Varnish
- Vector Cache — Redis, DiskANN
- Disk Cache — SQLite, LMDB
- Embedding Cache — Redis, Parquet + Index

11 BENEFITS OF MEMORY & CACHING

MEMORY BENEFITS

- Context continuity across sessions
- Personalized & relevant responses
- Better decision-making for agents
- Retain knowledge & experience
- Reduce redundant data processing
- Enable long-term learning

CACHING BENEFITS

- Reduced latency
- Lower API / compute cost
- Higher throughput & scalability
- Improved user experience
- Protects backend systems
- Better resource utilization

Best Practice

- Use memory for what should be remembered.
- Use caching for what can be recomputed but is expensive.
- Set appropriate TTL, eviction & invalidation policies.
- Monitor cache hit rate and memory growth.

12 MEMORY DESIGN PATTERNS

1. **Sliding Window (Short-Term)**
   Keep last N messages in memory.
2. **Summary Memory**
   Summarize long conversations and store summary.
3. **Hierarchical Memory**
   Short-term → Summary → Long-term store.
4. **Retrieval-Augmented Memory**
   Retrieve relevant memeries from Vector DB.
5. **User Profile Memory**
   Store persistent user facts and preferences.

13 HOW MEMORY IS STORED (DATA STRUCTURE)

Component	Description	Example
id	Unique identifier	uuid-1234
content / text	Original text / message / fact	"I prefer Italian food."
embedding	Vector representation	[0.12, -0.34, ...]
metadata	Additional info (source, time, user)	{"source": "chat", "user_id": "u1"}
created_at	Timestamp	2024-05-20T10:00:00Z
tags / type	Category or memory type	preference, fact

14 CACHE STRATEGIES

- **TTL (Time To Live)** — Expire items after a set time.
- **LRU (Least Recently Used)** — Evict least recently used items.
- **LFU (Least Frequently Used)** — Evict least frequently used items.
- **Write-Through** — Write to cache and persistent store.
- **Write-Back** — Write to cache, persist later.

15 THINGS TO WATCH OUT FOR

- Cache Staleness (outdated data)
- Memory Growth (unbounded)
- PII / Privacy in long-term memory
- Security of stored data
- Cache Invalidation complexity
- Over-caching unnecessary data

16 QUICK REFERENCE

Use Case	Use Memory	Use Cache
Maintain conversation	✓ Yes	✗ No
Personalization	✓ Yes	✗ No
Repeat queries	✗ No	✓ Yes
Expensive LLM calls	✗ No	✓ Yes
Tool/API responses	✗ No	✓ Yes
Static knowledge base	✓ Yes	✓ Yes

MEMORY = Knowledge & Experience | CACHING = Speed & Efficiency | Together = Smarter, Faster, Scalable RAG & Agents

EVALUATION METRICS FOR RAG

1 WHAT ARE EVALUATION METRICS?

Evaluation metrics are quantitative measures used to assess how well a RAG system retrieves relevant context and generates accurate, faithful, and useful answers.

EXAMPLE

Question: "When was Apple M2 chip released?"

Ground Truth Answer: "June 6, 2022"

Good RAG Output ✓
"Apple M2 chip was announced on June 6, 2022."

Bad RAG Output ✗
"Apple M2 chip was released in 2023." (incorrect)

Metrics help us automatically measure which system performed better and why.

2 HOW EVALUATION WORKS IN RAG

RAG has two main stages to evaluate:

User Question → Retrieve (Context) → Generate (Answer) → Final Answer

Retrieval Evaluation
Did we retrieve the right context?

Generation Evaluation
Did we generate the right answer using the context?

How it works:

- Provide a set of questions with ground truth answers and/or relevant documents.
- Run your RAG system to get retrieved contexts and final answers.
- Compare with ground truth using metrics.
- Aggregate scores to understand strengths/weaknesses.

3 TYPES OF EVALUATION IN RAG

RAG evaluation is typically split into two levels:

RETRIEVAL LEVEL
(How good is the search?)

- Checks if retrieved passages contain the relevant information.
- Independent of the LLM generation.

GENERATION LEVEL
(How good is the answer?)

- Checks quality, correctness, faithfulness, and usefulness of the generated answer.
- Dependent on the LLM and retrieved context.

You Need Both!

Good retrieval + Good generation = Great RAG

If retrieval fails → LLM can't answer correctly.
If generation fails → even good context won't help.

4 KEY METRICS IN RAG

A. RETRIEVAL METRICS

Evaluate the quality of retrieved documents/passages

Metric	What It Measures	How It Works	Formula / Notes
Precision@k	Of the top-k retrieved items, how many are relevant?	(# relevant in top-k) / k	$\text{Precision@k} = \frac{\lvert\text{Relevant} \cap \text{Top-k}\rvert}{k}$
Recall@k	Of all relevant items, how many were retrieved in top-k?	(# relevant in top-k) / (total # relevant)	$\text{Recall@k} = \frac{\lvert\text{Relevant} \cap \text{Top-k}\rvert}{\lvert\text{All Relevant}\rvert}$
MRR (Mean Reciprocal Rank)	How early is the first relevant item retrieved?	Average of reciprocal rank of the first relevant item	$\text{MRR} = \frac{1}{\text{rank}_1}$ (rank_1 = rank of first relevant item)
nDCG@k (Normalized DCG)	Considers rank and relevance of all retrieved items	Rewards relevant items higher up in the list	$\text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$
Hit Rate@k	Did we retrieve at least one relevant item in top-k?	1 if ≥1 relevant in top-k else 0	$\text{Hit Rate@k} \in \{0,1\}$

B. GENERATION METRICS

Evaluate the quality of the generated answer

Metric	What It Measures	How It Works	Scoring
Faithfulness	Is the answer faithful to the provided context?	Checks if all claims in the answer are supported by the context.	0 (not faithful) → 1 (fully faithful)
Answer Relevancy	Is the answer relevant to the question?	Measures semantic similarity between answer and question.	0 (not relevant) → 1 (highly relevant)
Context Relevancy	Are the retrieved contexts relevant to the question?	Measures how relevant retrieved passages are to the question.	0 → 1
Hallucination Rate	How much of the answer is not supported by the context?	Detects claims in the answer that are not present in the context.	Lower is better (0% best)
Correctness / Accuracy	Is the answer factually correct compared to ground truth?	Compare with reference answer using exact match or semantic similarity.	0 → 1 or Exact Match (0/1)
Answer Completeness	Does the answer cover all aspects of the question?	Checks if key points from ground truth are present.	0 → 1
Fluency	Is the answer well-written and easy to understand?	Use LLM or model-based scoring for grammar and fluency.	0 → 1

5 RETRIEVAL EXAMPLE (Precision@3)

Question: "When was Apple M2 chip released?"

Ground Truth Relevant Docs: {D2}

Top-3 Retrieved Docs: {D1, D2, D3}

Rank	Document	Relevant?
1	D1	✗
2	D2	✓
3	D3	✗

Precision@3 = 1/3 = 0.33
Recall@3 = 1/1 = 1.00
MRR = 1/2 = 0.50
Hit Rate@3 = 1

6 GENERATION EXAMPLE

Question: "When was Apple M2 chip released?"
Retrieved Context: "Apple M2 chip was announced on June 6, 2022."
Generated Answer: "Apple M2 chip was announced on June 6, 2022."
Ground Truth Answer: "June 6, 2022"

Faithfulness: 0.95 (All claims supported)
Answer Relevancy: 0.98 (Highly relevant)
Hallucination Rate: 0% (No unsupported claims)
Correctness: 1.00 (Matches ground truth)
Completeness: 0.90 (Covers key point)
Fluency: 0.97 (Well written)

7 COMMON EVALUATION PIPELINE

Dataset (Questions + Ground Truth) → Run RAG System → Collect Outputs → Compute Metrics → Analyze & Improve

Use a diverse evaluation dataset that covers different topics, question types, and difficulty levels.

8 WHAT MAKES A GOOD EVALUATION SET?

- **Diverse topics and domains** (technical, general, domain-specific)
- **Different question types** (factoid, why, how, comparison, list)
- **Includes hard/edge cases** (ambiguous, multi-hop, long context)
- **Has ground truth answers** and/or relevant documents
- **Sufficient size** (50–500+ samples minimum for reliable scores)

9 POPULAR FRAMEWORKS & TOOLS

DeepEval
LLM-powered evaluation framework for RAG. Supports metrics like faithfulness, relevancy, context recall, hallucination rate and more.
Features:
- Custom metrics
- LLM-as-a-judge
- Easy integration
Website: depeval.com

RAGAS
Specialized evaluation framework for RAG. Focus on faithfulness, answer relevancy, context precision/recall, and more.
Features:
- RAG-specific metrics
- Open source
- Easy to use
Website: ragas.io

Trulens Eval
Evaluate LLM apps and RAG pipelines with feedback functions and structural & semantic metrics.
Features:
- Hallucination detection
- Feedback functions
- Tracing & monitoring
Website: trulens.org

LangChain Evaluations
Built-in evaluation utilities for LangChain applications.
Features:
- Q&A evaluation
- LLM judges
- Custom evaluators
Website: python.langchain.com

OpenAI Evals
Open-source evals for LLM applications including RAG.
Features:
- Standard datasets
- LLM-based judges
- Extensible
Website: github.com/openai/evals

10 BEST PRACTICES

- Evaluate both retrieval and generation.
- Use multiple metrics, not just one.
- Track metrics over time (regression testing).
- Break down results by question type/domain.
- Use human evaluation for final validation.
- Continuously monitor in production (online evaluation).

11 BM25 (KEYWORD RETRIEVAL) – HOW IT WORKS

What is BM25?

BM25 (Best Match 25) is a ranking function used in keyword search to estimate the relevance of documents to a query based on term frequency, document frequency, and document length.

BM25 Score Formula

$$\text{score}(D,Q) = \sum_{i=1}^{n} IDF(q_i) \times \frac{f(q_i,D) \cdot (k_1+1)}{f(q_i,D) + k_1 \times \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

How BM25 Works (Steps)

1. Tokenize query and documents.
2. Compute term frequency (TF) in documents.
3. Compute document frequency (DF) for each term.
4. Calculate IDF (Inverse Document Frequency):
$$IDF(q_i) = \log \frac{N - df(q_i) + 0.5}{df(q_i) + 0.5}$$
where N = total documents, $df(q_i)$ = documents containing term q.
5. Compute BM25 score for each document.
6. Rank documents by BM25 score (highest first).

BM25 Architecture

Documents → Tokenization → Inverted Index (Postings List) → BM25 Scoring → Ranked Results

Where BM25 is Used?

Keyword search in search engines (e.g., Elasticsearch, OpenSearch)
Hybrid search (combined with vector search)
Lexical matching for exact term queries

12 KEY TAKEAWAY

Evaluation is essential to build reliable and trustworthy RAG systems.

Measure → Understand → Improve

- Retrieve the right context.
- Generate the right answer.
- Continuously evaluate and iterate.

Better evaluation → Better RAG.

OBSERVABILITY LAYER CHEAT SHEET
FOR RAG & AGENTS

Monitor. Debug. Evaluate. Improve.

1 WHAT IS OBSERVABILITY?

Observability is the ability to understand the internal state of your RAG or Agentic system by collecting, measuring and analyzing data from every component.

EXAMPLE

If an answer is incorrect, observability helps you find: Was it a retrieval issue? Prompt issue? LLM hallucination? Tool failure? Slow DB?

2 WHY OBSERVABILITY MATTERS

- Understand system behavior end-to-end
- Debug failures & improve accuracy
- Measure quality, cost, latency & usage
- Ensure reliability, safety & compliance
- Continuously improve with real data

Key Idea:

You can't improve what you can't observe.

3 OBSERVABILITY IN RAG PIPELINE

Track & Measure at Every Stage

What happened? → How well did it work? → Why did it happen?

4 OBSERVABILITY IN AGENTIC SYSTEMS

Observe Agent Behavior

Phan → Act → Observe → Reflect → Repeat

5 WHAT TO OBSERVE?

Layer	What to Observe	Examples
Input Layer	User queries, session info, intent, metadata	Query text, user id, region, device
Retrieval Layer	Search quality, results, metadata	Top-k docs, scores, filters, latency
Reranking Layer	Re-rank scores, selected docs	Before/after rank, score deltas
Generation Layer	LLM usage, output quality, faithfulness	Tokens, latency, model, answer, citations
Agent Layer	Decisions, tool calls, actions, observations	Plan, tool input/output, result status
System Layer	Latency, errors, throughput, cost, resource usage	P95 latency, error rate, CPU, cost

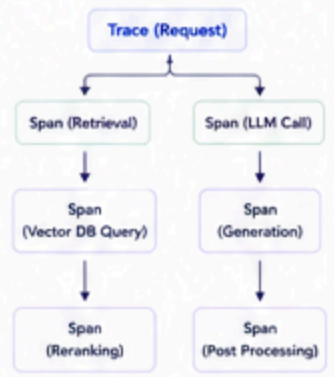
6 KEY OBSERVABILITY SIGNALS & METRICS

Category	Metrics	Description / Examples
Quality	Answer Relevancy	Human or LLM-as-a-judge score
Retrieval Effectiveness	Hit@k, Recall@k, nDCG	How good are retrieved docs
Faithfulness	Faithfulness, Hallucination Rate	Is the answer grounded?
Performance	Latency (P50/P95), Throughput	End-to-end or by component
Reliability	Error Rate, Timeout Rate	Failures across components
Cost	Input/Output Tokens, $ Cost	Per request / per user
Usage	Request Count, Active Users	Traffic, adoption, patterns
Safety	PII Leakage, Toxicity	Safety & compliance metrics

7 OBSERVABILITY DATA TO CAPTURE

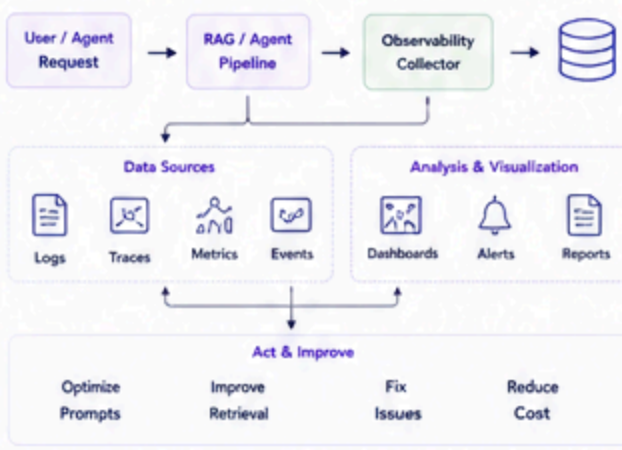
- User & Session Data
- Queries & Prompts
- Retrieved Documents & Scores
- Re-ranker Inputs/Outputs
- LLM Inputs (Prompt + Context)
- LLM Outputs (Answer + Citations)
- Tool Calls & Results (Agents)
- Errors, Exceptions & Stack Traces
- Latency, Tokens, Cost, Model Info
- User Feedback & Ratings

8 TYPES OF TRACES (EXAMPLE)



Trace = End-to-end request
Span = A step within the trace

9 OBSERVABILITY ARCHITECTURE



10 TOOLS & FRAMEWORKS

Tracing & Observability

LangSmith · Arize Phoenix · OpenTelemetry · Helicone · Langfuse · Traceloop

Monitoring & Logging

Prometheus · Grafana · Datadog · Sentry · Loki · ELK Stack

Evaluation & Quality

RAGAS · Evidently AI · Evidently A · Promptfoo

Vector DB Observability

Pinecone Observability · Weaviate Monitoring · Qdrant Telemetry · Milvus Monitoring

11 DASHBOARDS TO BUILD

Dashboard	Key Insights
Overview Dashboard	Total requests, latency, error rate, cost, quality score
Retrieval Dashboard	Hit@k, nDCG, top queries, filter impact
LLM Dashboard	Token usage, latency, model performance, cost
Agent Dashboard	Tool usage, success rate, steps per task, failure points
Quality Dashboard	Faithfulness, hallucination rate, user feedback trends
Cost Dashboard	Cost per request, per user, breakdown by model/tool

12 ALERTING EXAMPLES

- High error rate (> 2%)
- Latency P95 > 5s
- Drop in retrieval quality (nDCG < 0.6)
- Hallucination rate > 5%
- Spike in cost
- Tool failure rate > 5%

Notify via

Email · Slack · PagerDuty · Webhook

13 BEST PRACTICES

- Instrument every step (end-to-end tracing)
- Capture rich metadata (user, session, filters)
- Use consistent IDs (trace_id, session_id)
- Sample smartly (reduce overhead & cost)
- Monitor quality, not just performance
- Close the loop with evaluations & feedback
- Automate alerts & incident response
- Continuously review & iterate

14 END-TO-END OBSERVABILITY FLOW

15 EXAMPLE: OBSERVABILITY DATA (JSON)

```
{
  "trace_id": "trc_123abc",
  "session_id": "sess_799",
  "user_id": "user_01",
  "query": "What are the refund policies?",
  "retrieved_docs": [
    {"doc_id": "doc_1", "score": 0.87},
    {"doc_id": "doc_5", "score": 0.76}
  ],
  "rerank_top_k": 5,
  "selected_docs": ["doc_1", "doc_5", "doc_9"],
  "llm_model": "gpt-4o",
  "prompt_tokens": 1200,
  "completion_tokens": 350,
  "latency_ms": 2430,
  "cost_usd": 0.0123,
  "faithfulness_score": 0.92,
  "user_feedback": "thumbs_up"
}
```

16 EVALUATION & FEEDBACK LOOP

17 OBSERVABILITY MATURITY MODEL

Level	Stage	Description
L1	Basic	Logs & basic metrics, manual debugging
L2	Instrumented	Traces, dashboards, basic alerts
L3	Monitored	Proactive monitoring, quality metrics
L4	Optimized	Auto evaluations, closed-loop improvements
L5	Intelligent	AI-powered anomaly detection & self-healing

18 COMMON CHALLENGES

- Missing or inconsistent instrumentation
- High cardinality & noisy data
- Lack of ground truth for evaluation
- Observability overhead & cost
- Siloed tools & data
- No feedback loop

19 QUICK CHECKLIST

- Have you instrumented every step?
- Do you track quality metrics?
- Are dashboards & alerts in place?
- Do you capture user feedback?
- Are you monitoring cost & latency?
- Do you review & act regularly?

20 KEY TAKEAWAYS

Observe Everything
You can't fix what you don't observe.

Quality > Just Accuracy
Track faithfulness, relevancy & user value.

End-to-End Visibility
From user query to final answer.

Data + Evaluation + Action
Close the loop and continuously improve.

Better Observability = Better RAG & Agent Experiences

Observability turns your RAG & Agents from a black box into a transparent, measurable, and continuously improving system.